SESSION FIXATION ATTACK

Under the guidance of:
Prof. Preeti Soni

Panchagnula Vinutna
25CSM2S15

Date of Presentation:
01/11/2025

Introduction

- Session Fixation Attack - an attacker forces or tricks a user into using a session ID chosen by the attacker before the user logs in.
- After the user authenticates with that session ID, the attacker reuses the same ID to access the user's authenticated session.

Attacker Target Areas

Session management logic

Authentication flow & privilege change points

User delivery channels

Pre-analysis performed by attacker

- **Discover the target surface -** identify which pages, forms, and features exist. This shows where weaknesses might be.
- **Look for input points -** find places that accept user input (form fields, hidden fields) which could be used to inject or set values.
- **Test user-delivery channels -** see how easy it would be to get a user to click a link or accept a cookie (email, chat, redirects); this assesses the practicality of an attack.

Tools used by attacker

- **Web browser + DevTools (Chrome/Brave/Firefox) -** inspect cookies, session IDs, links and test clicking crafted URLs.
- **Burp Suite -** intercept and modify requests/responses, rewrite Set-Cookie/ URL parameters, and replay traffic.
- **URL shortener/ redirect page -** hide long malicious query strings and make links look legitimate when sending to victims.
- **QR-code generator -** create QR codes for mobile-based delivery of the malicious link.
- **HTTP proxy logs & server access -** examine server logs or session files to show the same SID being used pre- and post-login.

Tools and coding platforms used

- XAMPP
- PHP
- Visual Studio Code
- Brave Browser

Code

attacker_setup.php

```php
<?php
// attacker_setup.php
// Purpose: create a safe attacker-controlled SID and show crafted links.
// IMPORTANT: This is for lab/demo only.

$base = 'http://localhost/Session_Fixation_Attack';

// generate a safe session id (only allowed chars A-Z a-z 0-9 - ,)
$attacker_sid = 'ATTACKER-' . bin2hex(string: random_bytes(length: 6)); // safe hex
if (preg_match(pattern: '/^[A-Za-z0-9,-]+$/', subject: $attacker_sid) !== 1) {
    $attacker_sid = 'ATTACKER' . substr(string: bin2hex(string: random_bytes(length: 8)), offset: 0, length: 16);
}

// set the session id and start session
session_id(id: $attacker_sid);
session_start();
// set marker so a session file is created
$_SESSION['attacker'] = true;
?>
```

Code

login.php

```php
<?php
// login.php (vulnerable demo)
// Accepts ?sid= only if it matches allowed chars (keeps PHP happy) and demonstrates fixation.
// NOTE: In a real app you should NEVER accept session IDs from user input.

if (!empty($_GET['sid'])) {
    $incoming_sid = $_GET['sid'];
    if (preg_match(pattern: '/^[A-Za-z0-9,-]+$/', subject: $incoming_sid)) {
        session_id(id: $incoming_sid);
    }
}
session_start();

if (!empty($_SESSION['username'])) {
    header(header: "Location: protected.php");
    exit();
}
?>
```

Code

process_login.php

```php
<?php
// process_login.php (vulnerable)
// Accepts and sets SID only if safe; does NOT regenerate id after login (intentional vulnerability)

if (!empty($_GET['sid'])) {
    $incoming_sid = $_GET['sid'];
    if (preg_match(pattern: '/^[A-Za-z0-9,-]+$/', subject: $incoming_sid)) {
        session_id(id: $incoming_sid);
    }
}
session_start();

// Dummy authentication: any non-empty username is accepted (for demo only)
if (!empty($_POST['username'])) {
    // VULNERABLE: no session_regenerate_id here (so attacker SID remains valid)
    $_SESSION['username'] = $_POST['username'];
    header(header: "Location: protected.php");
    exit();
} else {
    echo "Login failed. <a href='login.php'>Try again</a>";
}

```

Code

protected.php

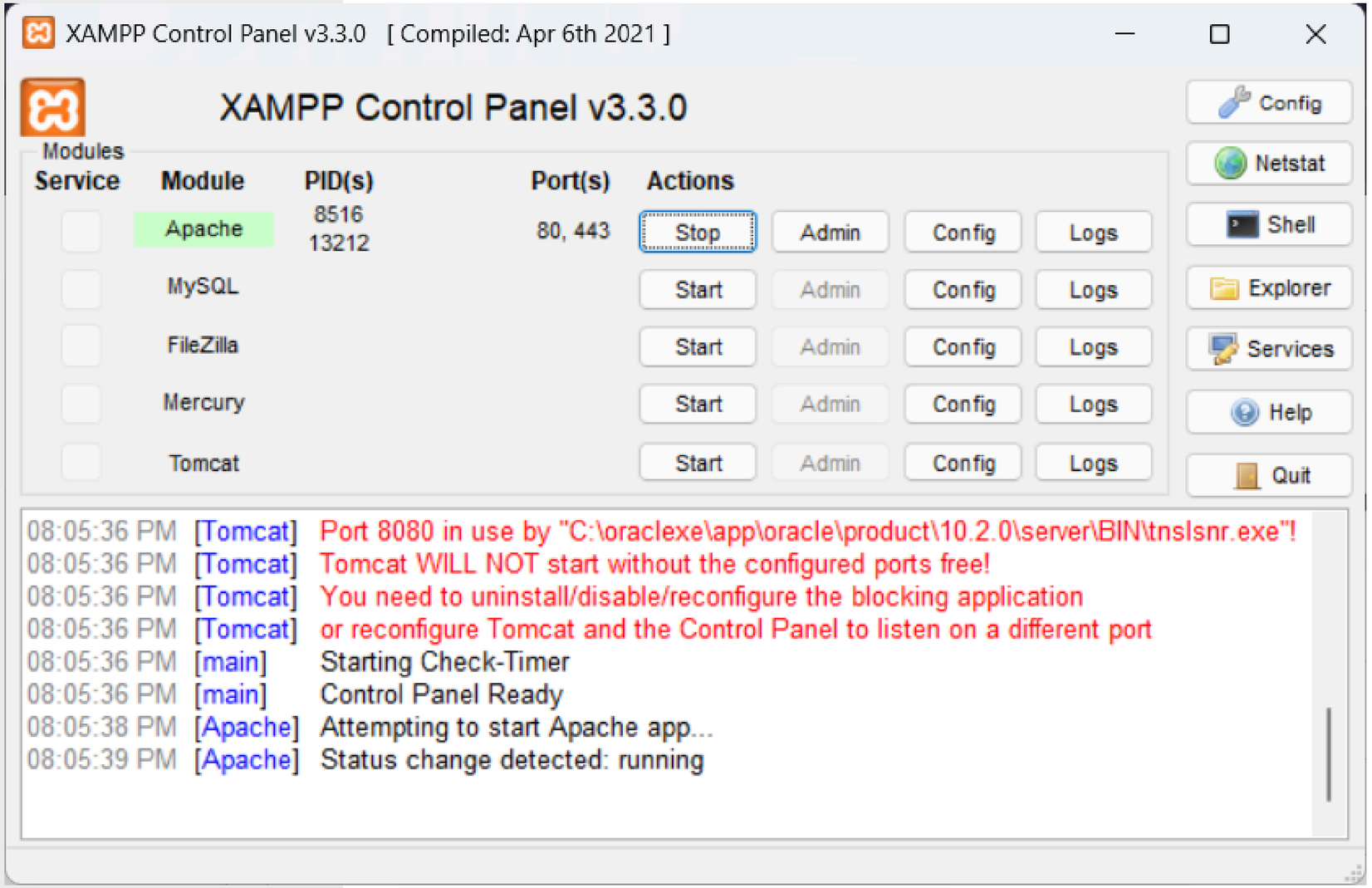
```php
<?php
// protected.php (vulnerable view)
// Accepts sid via GET (validated) to let attacker reuse the session id (demo only)
if (!empty($_GET['sid'])) {
    $incoming_sid = $_GET['sid'];
    if (preg_match(pattern: '/^[A-Za-z0-9,-]+$/', subject: $incoming_sid)) {
        session_id(id: $incoming_sid);
    }
}
session_start();

if (empty($_SESSION['username'])) {
    echo "<p>Not logged in. <a href='login.php'>Login (Vulnerable)</a></p>";
    exit();
}
?>
```

Code

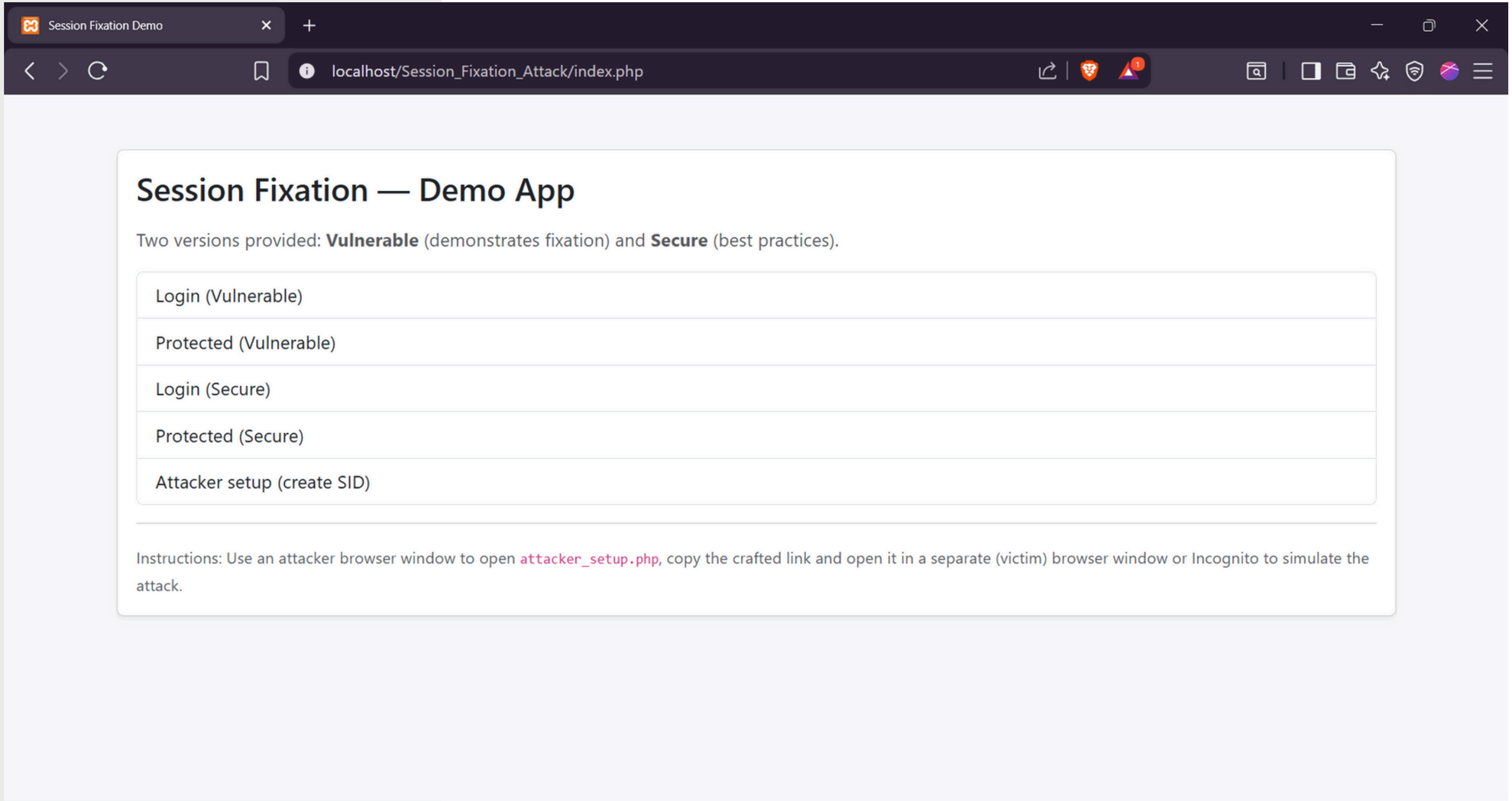
process_login_secure.php

```php
<?php
// process_login_secure.php - secure processing: regenerates session id after successful login

// Start session with secure parameters (optional improvement)
session_set_cookie_params(lifetime_or_options: [
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',      // leave default for localhost
    'secure' => false,   // set to true in production with HTTPS
    'httponly' => true,
    'samesite' => 'Lax'
]);
session_start();

if (!empty($_POST['username'])) {
    // Regenerate the session id after login to prevent fixation
    session_regenerate_id(delete_old_session: true);
    $_SESSION['username'] = $_POST['username'];
    header(header: "Location: protected_secure.php");
    exit();
} else {
    echo "Login failed. <a href='login_secure.php'>Try again</a>";
}

```

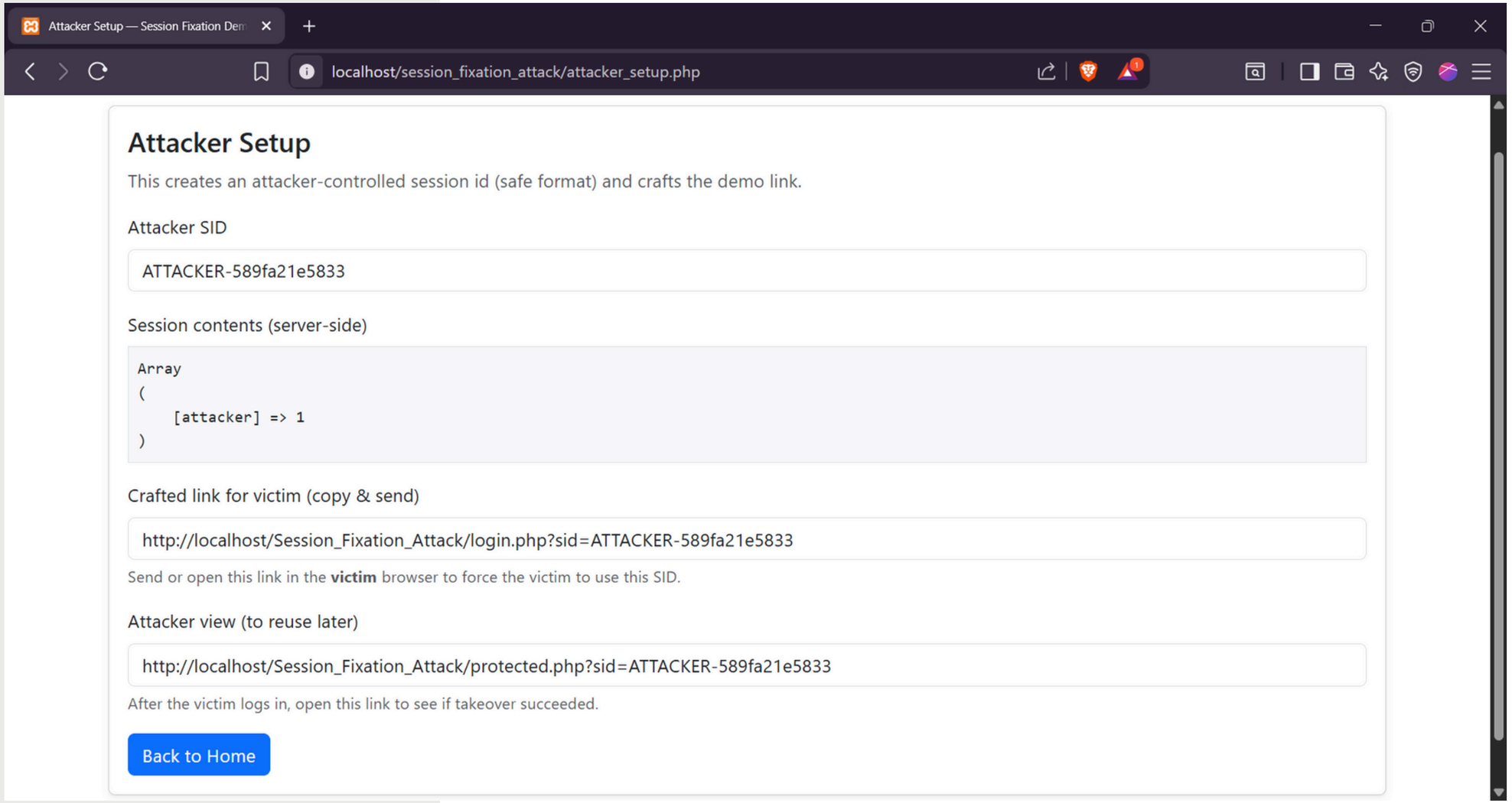
Step - 1


XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
8516
13212
80, 443
Stop
Admin
Config
Logs
MySQL
Start
Admin
Config
Logs
FileZilla
Start
Admin
Config
Logs
Mercury
Start
Admin
Config
Logs
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
08:05:36 PM [Tomcat] Port 8080 in use by "C:\oraclexe\app\oracle\product\10.2.0\server\BIN\tnslsnr.exe"!
08:05:36 PM [Tomcat] Tomcat WILL NOT start without the configured ports free!
08:05:36 PM [Tomcat] You need to uninstall/disable/reconfigure the blocking application
08:05:36 PM [Tomcat] or reconfigure Tomcat and the Control Panel to listen on a different port
08:05:36 PM [main] Starting Check-Timer
08:05:36 PM [main] Control Panel Ready
08:05:38 PM [Apache] Attempting to start Apache app...
08:05:39 PM [Apache] Status change detected: running

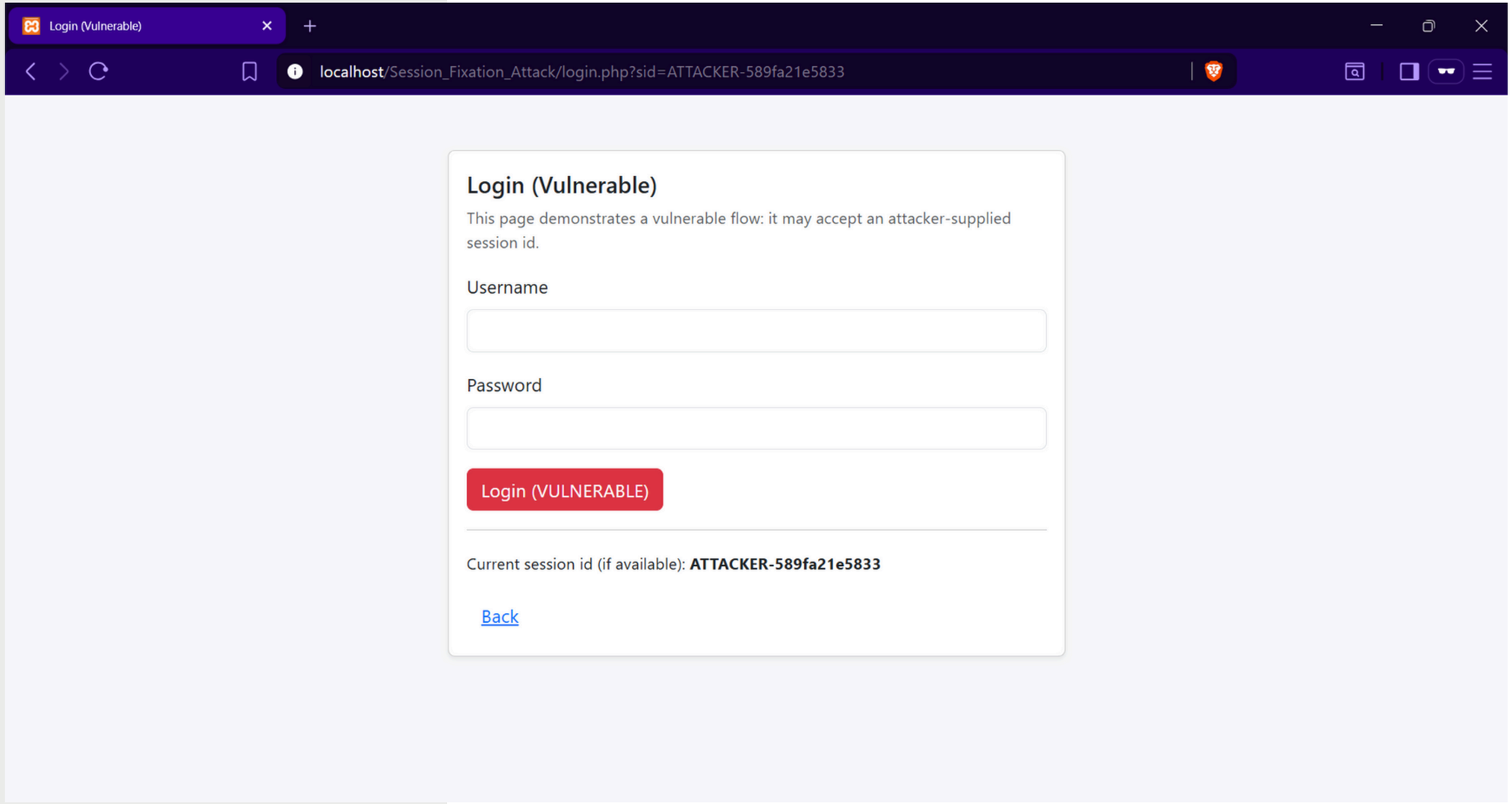
Step - 2


Session Fixation Demo
localhost/Session_Fixation_Attack/index.php
Session Fixation — Demo App
Two versions provided: Vulnerable (demonstrates fixation) and Secure (best practices).
Login (Vulnerable)
Protected (Vulnerable)
Login (Secure)
Protected (Secure)
Attacker setup (create SID)
Instructions: Use an attacker browser window to open attacker_setup.php, copy the crafted link and open it in a separate (victim) browser window or Incognito to simulate the attack.

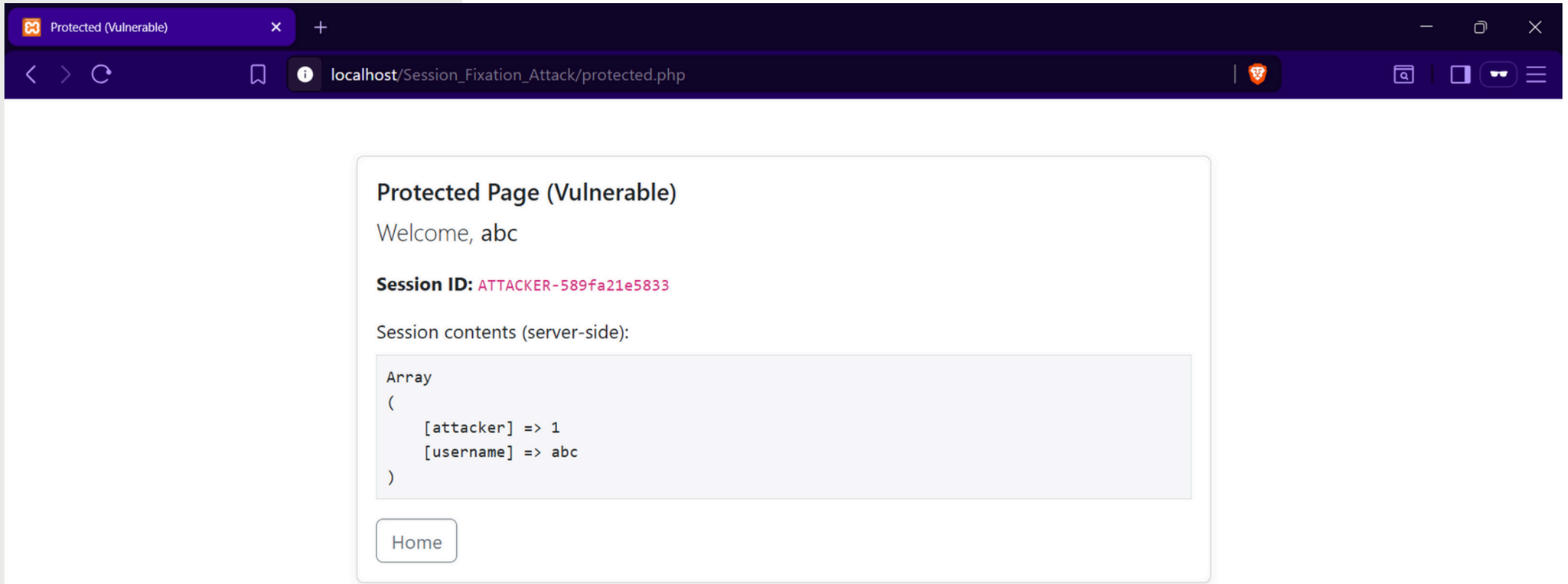
Step - 3


Attacker Setup — Session Fixation Dem
localhost/session_fixation_attack/attacker_setup.php
Attacker Setup
This creates an attacker-controlled session id (safe format) and crafts the demo link.
Attacker SID
ATTACKER-589fa21e5833
Session contents (server-side)
Array
(
[attacker] => 1
)
Crafted link for victim (copy & send)
http://localhost/Session_Fixation_Attack/login.php?sid=ATTACKER-589fa21e5833
Send or open this link in the victim browser to force the victim to use this SID.
Attacker view (to reuse later)
http://localhost/Session_Fixation_Attack/protected.php?sid=ATTACKER-589fa21e5833
After the victim logs in, open this link to see if takeover succeeded.
Back to Home

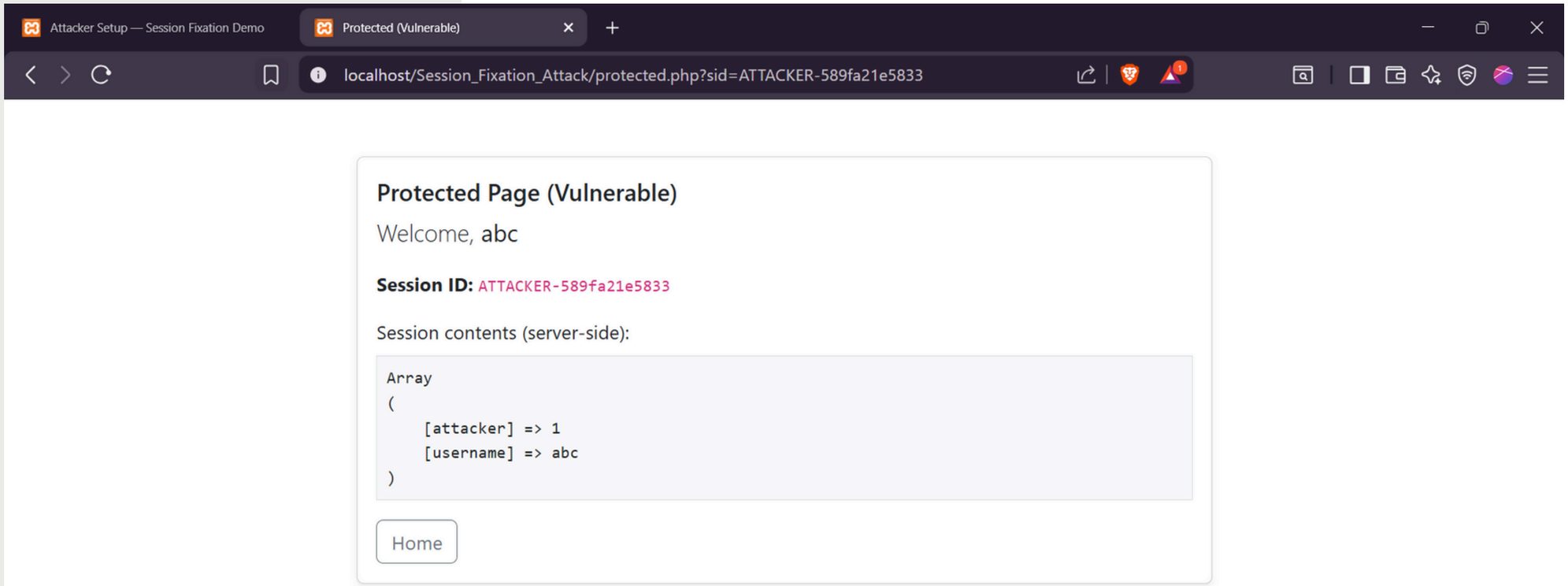
Step - 4


Login (Vulnerable)
localhost/Session_Fixation_Attack/login.php?sid=ATTACKER-589fa21e5833
Login (Vulnerable)
This page demonstrates a vulnerable flow: it may accept an attacker-supplied session id.
Username
Password
Login (VULNERABLE)
Current session id (if available): ATTACKER-589fa21e5833
Back

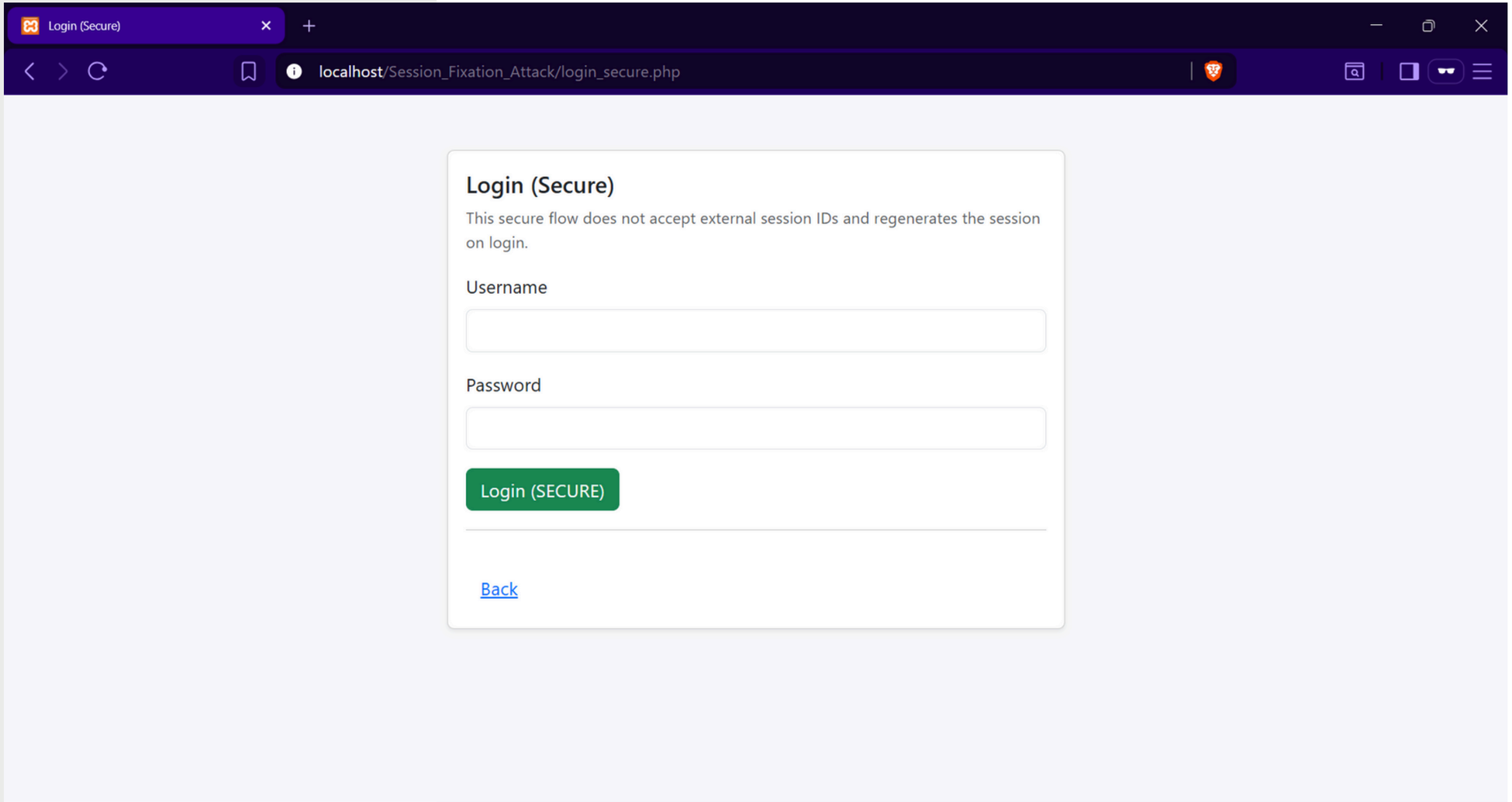
Step - 5


Protected (Vulnerable)
localhost/Session_Fixation_Attack/protected.php
Protected Page (Vulnerable)
Welcome, abc
Session ID: ATTACKER-589fa21e5833
Session contents (server-side):
Array
(
[attacker] => 1
[username] => abc
)
Home

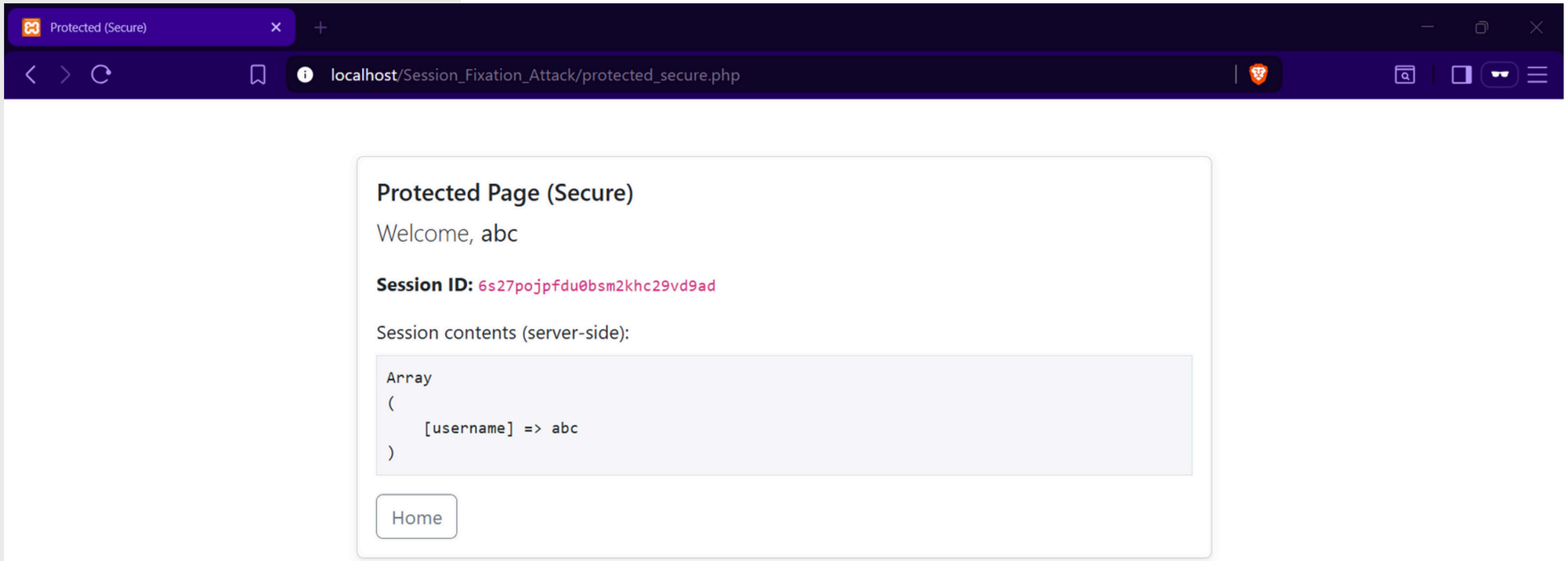
Step - 6


Attacker Setup — Session Fixation Demo
Protected (Vulnerable)
localhost/Session_Fixation_Attack/protected.php?sid=ATTACKER-589fa21e5833
Protected Page (Vulnerable)
Welcome, abc
Session ID: ATTACKER-589fa21e5833
Session contents (server-side):
Array
(
[attacker] => 1
[username] => abc
)
Home

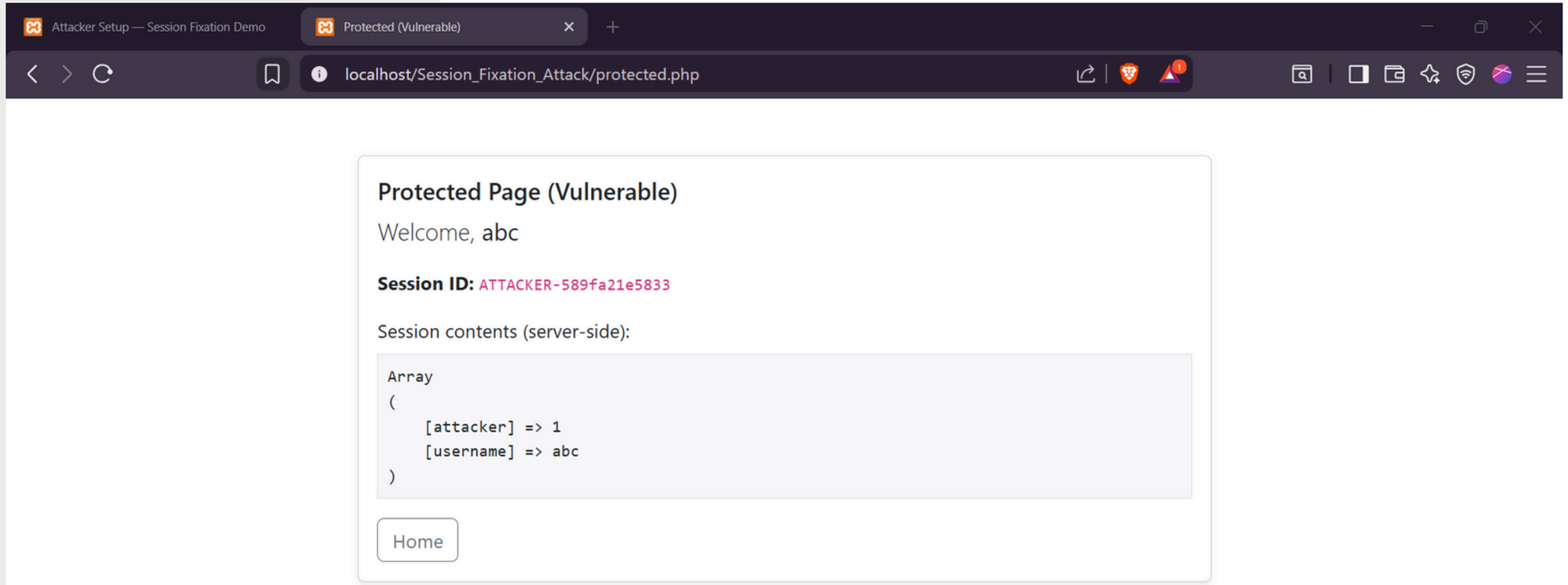
Security Measures



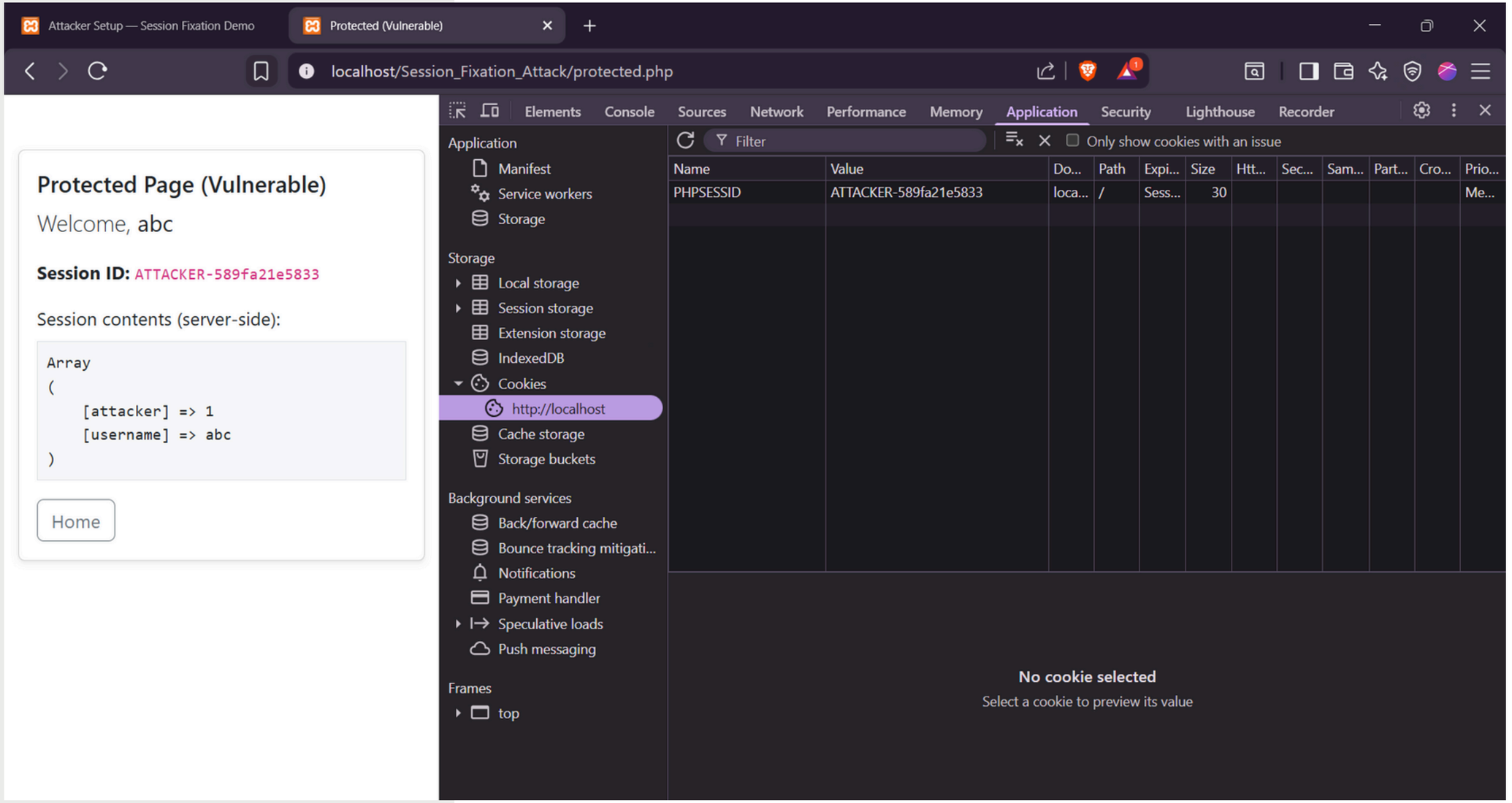
Security Measures



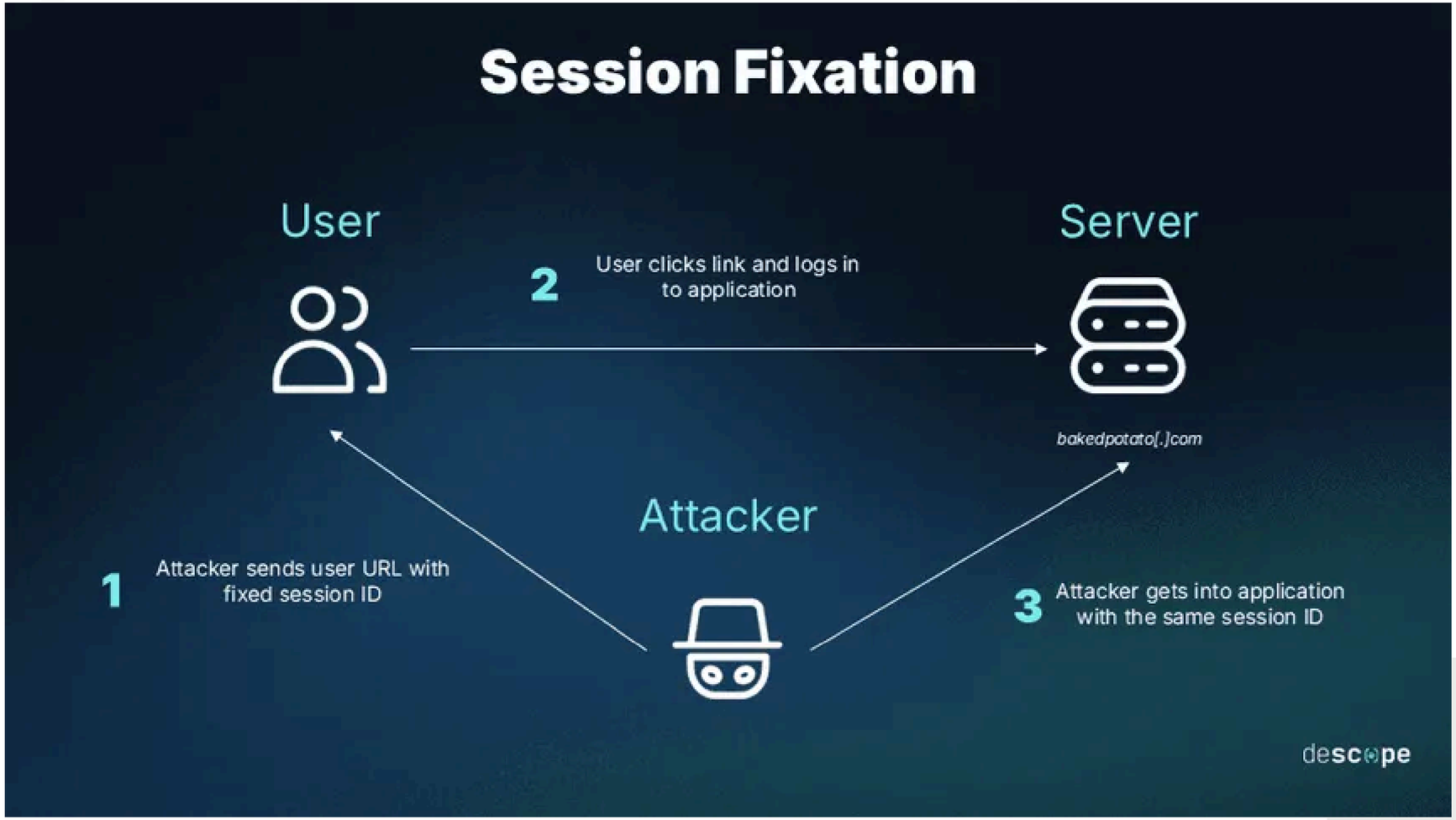
Security Measures



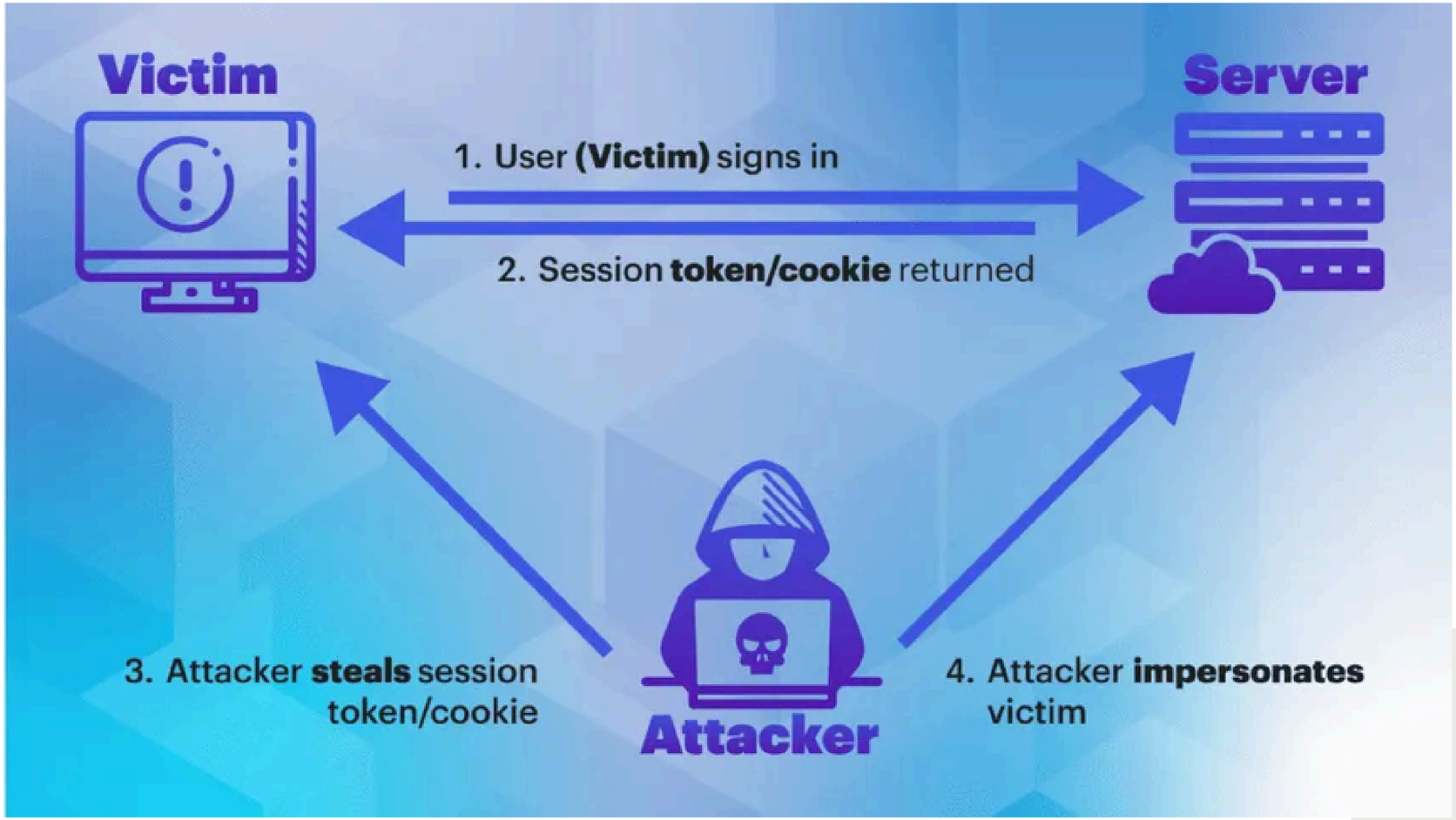
Security Measures



Flowchart – Session Fixation



Flowchart – Session Hijacking



Possible Security Patches to combat these attacks

- Regenerate Session ID after Login
- Do Not Accept Session IDs from URL or User Input
- Set Secure Cookie Attributes
- Use HTTPS for All Communication
- Implement Short Session Timeouts and Re-authentication

Modern/Variant forms of Session Fixation

Link-Shortener to Hide Fixation

JWT Fixation

Cookie Injection via XSS

Persistent “Remember Me” Token Fixation

THANK YOU